

LABORATORY OBSERVATION

Course	Full Stack Web Development
Course Code	23CM4121
Observation Task No.	8
Student Name	B.JATIN
Roll No.	A24126552070
Date	21-09-2026

1. TITLE

Express.js Middleware for Request Logging

2. AIM

To understand Express.js middleware and demonstrate middleware that logs incoming HTTP requests.

3. OBJECTIVES

1. Understand middleware in Express.js.
2. Understand the request-response cycle.
3. Create custom middleware.
4. Use next() to pass control to the next middleware or route.
5. Log HTTP methods and requested URLs.

4. THEORY

Express middleware is a function that has access to the request object, response object and next function. Middleware can inspect or modify requests and responses, perform logging or validation, and then call next() so processing continues. Application-level middleware registered with app.use() can run for matching incoming requests.

5. ALGORITHM / PROCEDURE

1. Create an Express application.
2. Define a requestLogger middleware function.
3. Read the HTTP method and URL from req.
4. Print the request information with a timestamp.
5. Call next() to continue request processing.
6. Register the middleware using app.use().
7. Create / and /about routes.
8. Start the server and send GET requests.

6. PROGRAM

Source file: server.js

```
// Task 8: Express.js middleware logging

const express = require("express");
const app = express();
const PORT = 3001;

// Logging middleware runs for every incoming request.
function requestLogger(req, res, next) {
  const time = new Date().toLocaleTimeString();
  console.log(`[${time}] ${req.method} ${req.url}`);
  next();
}

// Register the middleware before the routes.
app.use(requestLogger);

app.get("/", (req, res) => {
  res.send("Welcome to the Express.js Middleware Demo");
});

app.get("/about", (req, res) => {
  res.send("This application demonstrates request logging middleware.");
});

app.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
});
```

7. CODE / SYNTAX EXPLANATION

Code / Syntax	Explanation
function requestLogger(req,res,next)	Defines custom middleware.
req.method	Contains the incoming HTTP method.
req.url	Contains the requested URL.
next()	Passes control to the next middleware or route.
app.use(requestLogger)	Registers the middleware at application level.
app.get()	Defines GET request handlers.
res.send()	Sends a response to the client.
app.listen()	Starts the Express server.

8. TEST CASES AND EXECUTION DATA

Test Case	Input / Action	Expected Result	Status
TC-01	Start server	Server listens on port 3001	Pass
TC-02	GET /	Request is logged and response returned	Pass
TC-03	GET /about	Request is logged and response returned	Pass
TC-04	Send multiple requests	Each request produces a log entry	Pass

9. OUTPUT

Sample execution output. Install the required external package and run the program locally before final submission.

Task 8 - Execution Output

```
Server running at http://localhost:3001
[10:30:01 PM] GET /
[10:30:02 PM] GET /about

ROOT RESPONSE:
Welcome to the Express.js Middleware Demo

ABOUT RESPONSE:
This application demonstrates request logging middleware.
```

Output summary:

Server running at http://localhost:3001
[10:30:01 PM] GET /
[10:30:02 PM] GET /about

ROOT RESPONSE:
Welcome to the Express.js Middleware Demo

ABOUT RESPONSE:
This application demonstrates request logging middleware.

10. RESULT

Thus, Express.js middleware was successfully demonstrated by logging incoming requests and forwarding each request to the appropriate route using `next()`. The sample output shows the expected terminal log and route responses.